Detailed Metric Breakdown

This page provides a technical explanation of how each metric in the dashboard is calculated, including the underlying SQL formulas, data sources, and transformation logic.

Primary Dataset

All core KPIs are derived from a **local metric view** built on the source table:

Source Table: `coffeeshop.brightlearn.brightlearn_coffee_shop_2`

The metric view defines measures (pre-aggregated values) and dimensions (grouping fields) that power every widget on the dashboard. Each measure is computed using a specific SQL expression applied to the raw transaction data.

KPI Calculations

1. Total Revenue

Formula: `SUM(CAST(REPLACE(unit_price, ',', '.') AS DOUBLE) * transaction_qty)`

How it works:

- The `unit_price` column is stored as a **string** with European decimal format (comma separators, e.g., `"3,00"`)
- `REPLACE(unit_price, ',', '.')` converts the comma to a decimal point (e.g., `"3.00"`)
- `CAST(... AS DOUBLE)` converts the string to a numeric type
- The price is multiplied by `transaction_qty` (units purchased) for each transaction
- `SUM(...)` adds up the revenue across all transactions
- **Result:** R 698,812
- **Formatted as:** ZAR currency with 2 decimal places

---

2. Total Orders

Formula: `COUNT(DISTINCT transaction_id)`

How it works:

- Each transaction has a unique `transaction_id`
- `COUNT(DISTINCT ...)` counts the number of unique transaction IDs
- **Result:** 149,116 orders

---

3. Total Units Sold

Formula: `SUM(transaction_qty)`

How it works:

- `transaction_qty` represents the number of items in each transaction (typically 1 or 2)
- `SUM(...)` adds up all quantities across all transactions
- This is distinct from Total Orders — one order can contain multiple units

---

4. Average Order Value (AOV)

Formula: `MEASURE(revenue) / MEASURE(total_orders)`

How it works:

- Divides Total Revenue by Total Orders
- `MEASURE(revenue)` references the pre-defined revenue measure (calculation #1 above)
- `MEASURE(total_orders)` references the pre-defined order count (calculation #2 above)
- **Result:** R 698,812 ÷ 149,116 = R 4.69
- **Formatted as:** ZAR currency with 2 decimal places

---

5. Total Products

Formula: `COUNT(DISTINCT product_id)`

How it works:

- Each product has a unique `product_id`
- `COUNT(DISTINCT ...)` counts the number of unique product IDs
- **Result:** 80 distinct products across 9 categories

---

6. Total Stores

Formula: `COUNT(DISTINCT store_id)`

How it works:

- Each store has a unique `store_id`
- `COUNT(DISTINCT ...)` counts the number of unique store IDs
- **Result:** 3 stores (Hell's Kitchen, Astoria, Lower Manhattan)

Chart Calculations

Revenue by Month (Line Chart)

Formula:

- **X-axis:** `DATE_TRUNC('MONTH', transaction_date)` — truncates the timestamp to the first day of the month
- **Y-axis:** `MEASURE(revenue)` — the revenue measure grouped by month
- Groups all transactions by month and sums the revenue for each

---

Daily Revenue Trend (Line Chart)

Formula:

- **X-axis:** `DATE_TRUNC('DAY', transaction_date)` — truncates the timestamp to the date portion
- **Y-axis:** `MEASURE(revenue)` — the revenue measure grouped by day
- Groups all transactions by day and sums the revenue for each

---

Revenue by Product Category (Bar Chart)

Formula:

- **Category axis:** `product_category` — groups transactions by category
- **Value axis:** `MEASURE(revenue)` — sums revenue per category
- Sorted descending by revenue to show highest-performing categories first

---

Revenue by Store Location (Bar Chart)

Formula:

- **Category axis:** `store_location` — groups transactions by store
- **Value axis:** `MEASURE(revenue)` — sums revenue per store
- Sorted descending by revenue

Order Performance by Day of Week (Bar Chart)

Formula:

- **Category axis:** `DAYNAME(transaction_date)` — extracts the day name (Mon–Sun)
- **Value axis:** `MEASURE(total_orders)` — counts orders per day of week
- Groups all transactions by day name and counts distinct transactions

---

Top 10 Products by Revenue (Horizontal Bar)

Formula:

- **Category axis:** `product_detail` — groups by full product name
- **Value axis:** `MEASURE(revenue)` — sums revenue per product
- Sorted descending by revenue, limited to top 10 visible categories (scrollable)

Top 10 Products by Quantity (Horizontal Bar)

Formula:

- **Category axis:** `product_detail` — groups by full product name
- **Value axis:** `MEASURE(total_units)` — sums quantity per product
- Sorted descending by quantity, limited to top 10 visible categories

---

Product Sales Summary (Table)

Formula:

- Groups by `product_detail`, `product_category`, and `store_location`
- Columns: product name, category, store, `MEASURE(revenue)`, `MEASURE(total_units)`, `MEASURE(total_orders)`
- All three grouping dimensions combined, showing revenue/units/orders per product per store

---

Secondary Dataset: Spending Tiers

A separate SQL dataset powers the Revenue by Spending Tier chart:

Spending Tier Classification:

```
CASE
  WHEN transaction_qty * CAST(REPLACE(unit_price, ',', '.') AS DOUBLE) <= 3.0  THEN 'Low'
  WHEN transaction_qty * CAST(REPLACE(unit_price, ',', '.') AS DOUBLE) <= 6.0  THEN 'Medium'
  ELSE 'High'
END
```

How it works:

- Each transaction's total spend is calculated as `quantity × unit_price`
- Transactions are classified into three tiers:
  - **Low:** Total spend ≤ R 3.00
  - **Medium:** Total spend ≤ R 6.00
  - **High:** Total spend > R 6.00
- The chart sums `transaction_spend` for each tier and displays as a bar chart

---

Derived Dimensions

Dimension	Formula	Used In
`transaction_day`	`DATE(transaction_date)`	Daily Revenue Trend chart